Webfejlesztés – gyakorló feladatsor

C# Web API (15 pont) · Webdizájn (10 pont) · PHP backend (15 pont) – összesen 40 pont

A feladatsor mindhárom része egy kitalált kávézólánc, a **Babszem Kávézó** köré épül. A három rész egymástól független, tetszőleges sorrendben megoldható. Minden részhez a **Források** mappában találja a szükséges állományokat (adatbázis-szkriptek, képek, szövegek, kész frontend).

A források szerkezete

```
Források/
├── 1_API/
│   └── BabszemDB_MSSQL.sql          ← MS SQL Server adatbázis (C# API)
├── 2_Webdizajn/
│   ├── kepek/                       ← hero.jpg, rolunk1-2.jpg, kave1-3.jpg
│   └── szoveg.txt                   ← a weboldal összes szövege
└── 3_PHP/
    ├── babszem_webshop.sql          ← MySQL/MariaDB adatbázis (XAMPP)
    └── babszem_webshop/
        ├── frontend/                ← KÉSZ webshop felület (html, css, js)
        └── backend/                 ← ÜRES – ide kell elkészítenie a PHP fájlokat
```

Általános tudnivalók

- A szövegeket, képeket ne gépelje/rajzolja újra, használja a megadott forrásokat!
- Az API-k minden válasza JSON formátumú legyen. A JSON-mezők nevei pontosan egyezzenek a mintákkal (kis- és nagybetűre is)!
- A mintaválaszok a megadott adatbázis eredeti tartalmára vonatkoznak. A tizedes számok formátuma (pl. `590` vagy `590.00`) eltérhet, ez nem hiba.
- Az API-kat Swaggerrel, Postmannel vagy a VS Code Thunder Client bővítményével is tesztelheti.
- A feladat értékelésekor csak a működő, a leírásnak megfelelő megoldás ér pontot.

I. rész – C# Web API fejlesztése (15 pont)

A Babszem Kávézó négy városban működő üzleteinek forgalmi adatait egy Microsoft SQL Server adatbázis tárolja. Készítsen ASP.NET Core Web API alkalmazást, amely az adatbázis adatait REST végpontokon keresztül elérhetővé teszi!

Az adatbázis létrehozása

Nyissa meg a **Források/1_API/BabszemDB_MSSQL.sql** állományt SQL Server Management Studióban (vagy a Visual Studio SQL Server Object Explorerében), és futtassa le! A szkript létrehozza a **BabszemDB** adatbázist, a táblákat és feltölti őket adatokkal (8 kategória, 32 termék, 4 üzlet, 240 rendelés, 477 rendelési tétel).

Az adatbázis táblái

Tábla	Mezők	Megjegyzés
Categories	CategoryID (PK), CategoryName, Description	termékkategóriák
Shops	ShopID (PK), ShopName, City, Address	a kávézólánc üzletei
Products	ProductID (PK), CategoryID (FK), ProductName, UnitPrice (decimal), StockQuantity	termékek
Orders	OrderID (PK), ShopID (FK), OrderDate (datetime), PaymentMethod ('Cash' / 'Card')	rendelések (blokkok)
OrderItems	ItemID (PK), OrderID (FK), ProductID (FK), Quantity, SalePrice (decimal)	rendelési tételek

Kapcsolatok: *Categories 1 – N Products*, *Shops 1 – N Orders*, *Orders 1 – N OrderItems*, *Products 1 – N OrderItems*. Egy rendelési tétel értéke: *Quantity · SalePrice*.

Feladatok

1. **(2 pont)** Hozzon létre egy **BabszemAPI** nevű ASP.NET Core Web API projektet (.NET 8)! Telepítse a *Microsoft.EntityFrameworkCore.SqlServer* és *Microsoft.EntityFrameworkCore.Tools* NuGet csomagokat, a kapcsolati karakterláncot *BabszemDB* néven az *appsettings.json*-ban tárolja! Készítse el az entitásosztályokat és a **BabszemDbContext** osztályt (kézzel vagy a Package Manager Console-ban a `Scaffold-DbContext` paranccsal), és regisztrálja a kontextust a *Program.cs*-ben! Az API Swaggerrel legyen tesztelhető.
2. **(1 pont)** `GET /api/categories` – az összes kategória azonosító szerint rendezve, a kategóriába tartozó termékek számával (*productCount*) együtt.
3. **(2 pont)** `GET /api/products` – a termékek listája név szerint rendezve (*productId, productName, categoryName, unitPrice, stockQuantity*). Két elhagyható lekérdezési paramétert fogadjon: `categoryId` esetén csak az adott kategória termékeit, `search` esetén csak azokat, amelyek nevében a keresett szöveg (kis- és nagybetűtől függetlenül) szerepel. A két szűrő együtt is használható.
4. **(1 pont)** `GET /api/products/{id}` – egy termék adatai a kategória azonosítójával és nevével. Ha nincs ilyen termék: **404** és `{"message": "A termék nem található."}`
5. **(2 pont)** `GET /api/orders/{id}` – egy rendelés adatai: az üzlet neve, a dátum, a fizetési mód, a tételek listája (*productName, quantity, salePrice, lineTotal*) és a rendelés végösszege (*total*). Ha nincs ilyen rendelés: **404**, üzenet: *„A rendelés nem található.”*
6. **(2 pont)** `GET /api/shops/{id}/summary` – egy üzlet összesítője: név, város, a rendelések száma, ezen belül a készpénzes (*Cash*) és a kártyás (*Card*) rendelések száma, valamint a teljes bevétel (*revenue*). Ha nincs ilyen üzlet: **404**, üzenet: *„Az*

üzlet nem található.”

7. **(2 pont)** `GET /api/stats/top-products?top=5` – a legtöbb eladott darabszámú termékek (*productName, totalQuantity, totalRevenue*). A *top* paraméter alapértéke 5. Azonos darabszám esetén a nagyobb bevételű termék kerüljön előrébb! Ha *top* < 1: **400**, üzenet: *„A top értéke legalább 1 legyen.”*
8. **(2 pont)** `POST /api/products` – új termék felvétele (a kérés törzsét lásd lent). Ellenőrizze a bemenetet, és hiba esetén **400**-as kóddal az első talált hibát adja vissza: *„A termék neve kötelező.”*, *„Az ár csak pozitív szám lehet.”*, *„A készlet nem lehet negatív.”*, *„Nem létező kategória.”* Sikeres mentéskor **201 Created** kóddal adja vissza az új terméket (ugyanabban a formában, mint a 4. feladat).
9. **(1 pont)** `DELETE /api/products/{id}` – termék törlése. Nem létező terméknél **404**; ha a termék már szerepel valamelyik rendelésben, **409 Conflict** és *„A termék nem törölhető, mert már szerepel rendelésben.”*; sikeres törléskor **204 No Content**.

A végpontok összefoglalása

Metódus	Útvonal	Siker	Hibakódok
GET	/api/categories	200	–
GET	/api/products?categoryId=&search=	200	–
GET	/api/products/{id}	200	404
GET	/api/orders/{id}	200	404
GET	/api/shops/{id}/summary	200	404
GET	/api/stats/top-products?top=	200	400
POST	/api/products	201	400
DELETE	/api/products/{id}	204	404, 409

Mintaválaszok

`GET /api/categories` (részlet):

```
[
  {
    "categoryId": 1,
    "categoryName": "Espresso italok",
    "description": "Klasszikus feketekávék",
    "productCount": 4
  },
  {
    "categoryId": 2,
    "categoryName": "Tejes kávék",
    "description": "Cappuccino, latte és társaik",
    "productCount": 4
  },
  …
```

`GET /api/products?categoryId=2` → 200

```
[
  {
    "productId": 6,
    "productName": "Caffe latte",
    "categoryName": "Tejes kávék",
    "unitPrice": 950.00,
    "stockQuantity": 127
  },
  {
    "productId": 5,
    "productName": "Cappuccino",
    "categoryName": "Tejes kávék",
    "unitPrice": 890.00,
    "stockQuantity": 18
  },
```

…

GET /api/products/5 → 200, illetve GET /api/products/99 → 404


```
{
  "productId": 5,
  "productName": "Cappuccino",
  "categoryId": 2,
  "categoryName": "Tejes kávék",
  "unitPrice": 890.00,
  "stockQuantity": 18
}

{
  "message": "A termék nem található."
}
```


GET /api/orders/7 → 200


```
{
  "orderId": 7,
  "shopName": "Babszem Nagyerdő",
  "orderDate": "2026-03-02T15:27:50",
  "paymentMethod": "Cash",
  "items": [
    {
      "productName": "Espresso",
      "quantity": 1,
      "salePrice": 590.00,
      "lineTotal": 590.00
    },
    {
      "productName": "Frappé",
      "quantity": 1,
      "salePrice": 1190.00,
      "lineTotal": 1190.00
    },
    {
      "productName": "Flat white",
      "quantity": 2,
      "salePrice": 990.00,
      "lineTotal": 1980.00
    }
  ],
  "total": 3760.00
}
```


GET /api/shops/1/summary → 200


```
{
  "shopName": "Babszem Belváros",
  "city": "Budapest",
  "orderCount": 103,
  "cashOrders": 53,
  "cardOrders": 50,
  "revenue": 313450.00
}
```


GET /api/stats/top-products?top=3 → 200


```
[
  {
    "productName": "Caffe latte",
    "totalQuantity": 56,
    "totalRevenue": 53200.00
  },
  {
    "productName": "Espresso",
    "totalQuantity": 55,
    "totalRevenue": 32450.00
  },
  {
    "productName": "Cappuccino",
    "totalQuantity": 49,
    "totalRevenue": 43610.00
  }
]
```

POST `/api/products` – a kérés törzse és a válasz (201 Created):

```
{
  "categoryId": 8,
  "productName": "Pisztáciás latte",
  "unitPrice": 1290,
  "stockQuantity": 40
}
```

```
{
  "productId": 33,
  "productName": "Pisztáciás latte",
  "categoryId": 8,
  "categoryName": "Szezonális",
  "unitPrice": 1290,
  "stockQuantity": 40
}
```

Hibás kérésekre adott válaszok (400 / 409):

```
{
  "message": "A termék neve kötelező."
}
{
  "message": "Nem létező kategória."
}
{
  "message": "A termék nem törölhető, mert már szerepel rendelésben."
}
```

Ellenőrzéshez: a top 5 termék darabszáma rendre 56, 55, 49, 41, 40; az 5. helyen az Americano és a Vajas croissant is 40 db-ot ért el, de az Americano bevétele nagyobb (27 600 > 23 600). Egyetlen törölhető termék van: a 32-es azonosítójú Gesztenyés torta, amely még egyetlen rendelésben sem szerepelt.

II. rész – Webdizájn (10 pont)

Készítse el a Babszem Kávézó egyoldalas, reszponzív bemutatkozó weboldalát a minták alapján! A szövegeket a **Források/2_Webdizajn/szoveg.txt**, a képeket a **kepek** mappa tartalmazza. A megoldás három állományból álljon: **index.html**, **style.css** és **script.js**, a kepek mappa ezek mellett legyen. Külső CSS-keretrendszer (pl. Bootstrap) nem használható.

Színek és betűtípusok

Név (CSS változó)	Szín	Felhasználás
--sotet	#2b1d16	fejléc, lábléc háttere, szövegszín, kiemelt gomb
--krem	#f5ebe0	az oldal háttere, gombok háttere
--karamell	#c08552	menüsor háttere, dobozok kerete, árak, lábléc címei
--tejes	#e6d5c3	a Rólunk dobozok háttere

Címsorok (h1, h2, h3): **Georgia**, serif · Szövegtörzs: **Arial**, sans-serif. A színeket CSS változóként (`:root`) adja meg!

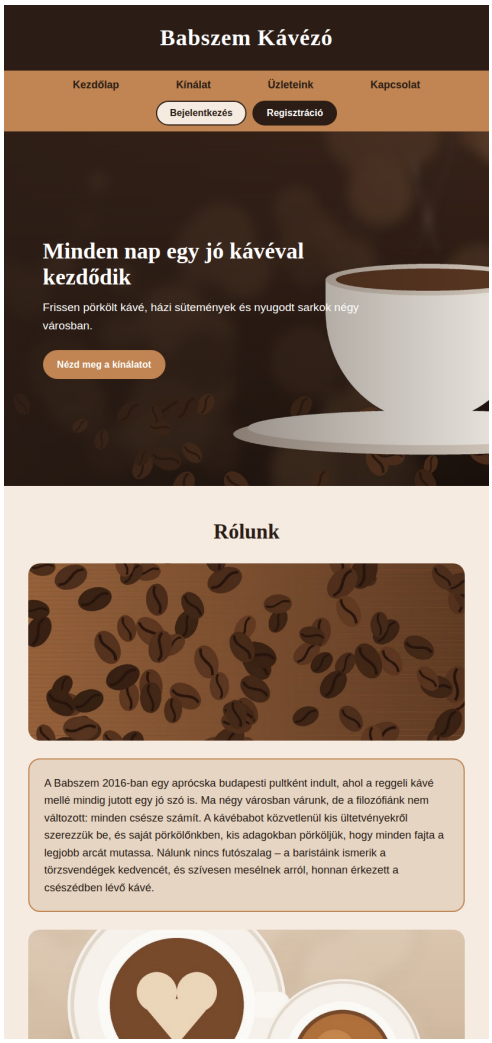
Feladatok

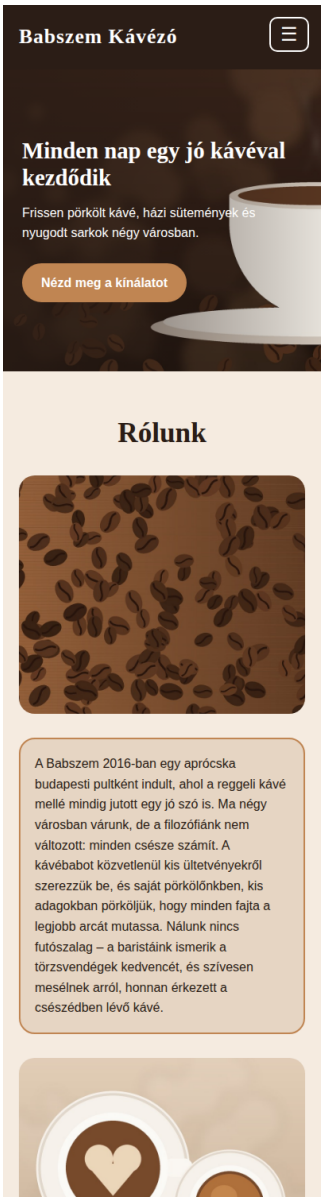
1. **(1 pont)** Szerkezet: szemantikus elemek (*header, nav, main, section, article, footer*), helyes *lang*, *charset*, *viewport* és *title*; minden képnek legyen *alt* szövege.
2. **(2 pont)** Fejléc: sötét háttéren középre igazított főcím. Alatta a karamell színű menüsor CSS griddel két részre osztva (2fr 1fr): bal oldalon a négy menüpont egyenletesen elosztva (flex), jobb oldalon a két lekerekített gomb (border-radius: 30px). A *Regisztráció* gomb sötét hátterű, fehér feliratú.
3. **(1 pont)** Hero szakasz: 600 px magas, a *hero.jpg* háttérképpel (cover, középre igazítva), amelyre balról jobbra halványuló sötét átmenet (linear-gradient) kerül. Bal oldalon a cím, az alcím és egy karamell színű, lekerekített gomb.
4. **(2 pont)** Rólunk szakasz: az oldal 80%-át kitöltő, középre igazított tartalom. Az első sorban a kép és a szövegdoboz 1fr 2fr, a második sorban 2fr 1fr arányú gridben áll. A képek kitöltik a helyüket (object-fit: cover), a dobozok lekerekítettek (20 px), karamell keretesek és tejes hátterűek.
5. **(1 pont)** Kedvenceink: három egyforma szélességű kártya (grid). A kártyák fehérek, lekerekítettek, árnyékosak, a kép 200 px magas. Ha az egérrel a kártya fölé visszük, kissé emelkedjen meg (transform).
6. **(1 pont)** Lábléc: sötét háttéren három egyforma oszlop (Címünk, Nyitvatartás, Elérhetőség), karamell színű címekkel; alatta vékony karamell vonallal elválasztva, középre igazítva a szerzői jogi sor.
7. **(2 pont)** Reszponzivitás:
   – **1000 px** alatt a menüsor egy oszlopos legyen (a gombok középen), a Rólunk sorai egymás alá kerüljenek úgy, hogy **mindkét sorban a kép legyen felül**, a kártyák két oszlopba rendeződjenek;
   – **600 px** alatt a menüsor rejtett, helyette a fejléc jobb felső sarkában egy ☰ (hamburger) gomb jelenik meg, amelyre kattintva a menü egymás alatti elemekkel lenyílik, illetve újabb kattintásra eltűnik (a *script.js*-ben egy *active* osztály ki-be kapcsolásával). A hero 380 px magas, a kártyák és a lábléc oszlopai egymás alá kerülnek.

Minta – asztali nézet (1400 px)



Minta – tablet (820 px), mobil (400 px) és mobil nyitott menüvel







A tablet és a mobil nézetből csak az oldal felső része látható; a további szakaszok a leírás szerint rendeződnek egymás alá.

III. rész – PHP backend fejlesztése (15 pont)

A Babszem online webshopjának felülete (frontend) **már elkészült**, de hiányzik mögüle a szerveroldal. Készítse el PHP nyelven, PDO segítségével azt a backendet, amelyre a kész frontend épül!

Előkészítés

- Indítsa el a XAMPP-ban az **Apache** és a **MySQL** szolgáltatást!
- phpMyAdminban importálja a **Források/3_PHP/babszem_webshop.sql** állományt! (Létrejön a *babszem_webshop* adatbázis 6 kategóriával és 16 termékkel.)
- Másolja a **babszem_webshop** mappát a *xampp/htdocs* mappába! A PHP fájlokat a **backend** almappába készítse el.
- A kész felület a `http://localhost/babszem_webshop/frontend/index.html` címen érhető el. A frontend a `../backend/termekek.php` és a `../backend/kategoriak.php` végpontokat hívja – a fájlneveket ezért pontosan így adja meg!

Az adatbázis táblái

Tábla	Mezők
categories	id (PK, AUTO_INCREMENT), name
products	id (PK, AUTO_INCREMENT), name, description (lehet NULL), price (egész, Ft), stock (egész, db)
product_categories	product_id (FK → products.id), category_id (FK → categories.id) – kapcsolótábla, egy termék több kategóriába is tartozhat

Feladatok

1. **(1 pont) config.php**: hozzon létre PDO kapcsolatot a *babszem_webshop* adatbázishoz *utf8mb4* karakterkódolással úgy, hogy hiba esetén kivételt dobjon (*PDO::ERRMODE_EXCEPTION*)! A többi fájl ezt töltse be (*require_once*).
2. **(1 pont) kategoriak.php**: GET kérésre adja vissza az összes kategóriát (*id, name*) név szerint rendezve. Más metódusra **405** és `{"error": "Method not allowed"}`.
3. **(4 pont) termekek.php**, GET paraméter nélkül: az összes termék azonosító szerint rendezve. Minden terméknél egy *categories* tömbben szerepeljenek a kategóriái (*id, name*) – ehhez kapcsolja össze a három táblát (LEFT JOIN), és a sorokat termékenként csoportosítsa! A kategória nélküli termék üres tömböt kapjon. Az *id, price, stock* értékek számként (ne szövegként) jelenjenek meg a JSON-ban.
4. **(2 pont) termekek.php?id=N**, GET: egyetlen termék ugyanebben a formában (objektumként, nem tömbben). Ha nincs ilyen: **404** és `{"error": "A termék nem található."}`. Az azonosítót előkészített lekérdezéssel (prepared statement) használja!
5. **(3 pont) termekek.php**, POST: új termék felvétele a kérés törzsében érkező JSON alapján (*name, description, price, stock, categories* – az utóbbi kategóriaazonosítók tömbje). Ellenőrzések és üzenetek (**400**): *„A név megadása kötelező.”*, *„Az ár csak pozitív egész szám lehet.”*, *„A készlet nem lehet negatív.”*; nem létező kategória esetén *„Nem létező kategória.”* A termék és a kategóriakapcsolatok mentése **tranzakcióban** történjen. Siker esetén **201** és a minta szerinti válasz.
6. **(2 pont) termekek.php?id=N**, PATCH: a termék készletének módosítása (`{"stock": 15}`). Negatív vagy hiányzó érték: **400**; nem létező termék: **404**; siker: **200** a minta szerint.
7. **(1 pont) termekek.php?id=N**, DELETE: a termék törlése (előbb a kapcsolótábla sorait kell törölni!). Nem létező termék: **404**; siker: **200** és `{"message": "Termék`

törölve."}.

8. **(1 pont)** A *termekek.php* minden válasza `Content-Type: application/json; charset=utf-8` fejléccel menjen ki; PATCH és DELETE esetén hiányzó *id* paraméterre **400** és *„Hiányzó azonosító.”*, egyéb metódusra (pl. PUT) **405** és *„Method not allowed”* legyen a válasz.

A végpontok összefoglalása

Metódus	URL	Törzs	Siker	Hibák
GET	backend/kategoriak.php	–	200	405
GET	backend/termekek.php	–	200	–
GET	backend/termekek.php?id=2	–	200	404
POST	backend/termekek.php	name, description, price, stock, categories	201	400
PATCH	backend/termekek.php?id=6	stock	200	400, 404
DELETE	backend/termekek.php?id=17	–	200	400, 404
egyéb	backend/termekek.php	–	–	405

Mintaválaszok

`GET backend/kategoriak.php` → 200

```
[
  {
    "id": 6,
    "name": "Ajándékötlet"
  },
  {
    "id": 5,
    "name": "Bio"
  },
  {
    "id": 3,
    "name": "Kapszula"
  },
  {
    "id": 4,
    "name": "Kiegészítők"
  },
  {
    "id": 2,
    "name": "Őrölt kávé"
  },
  {
    "id": 1,
    "name": "Szemes kávé"
  }
]
```

GET backend/termekek.php?id=2 → 200 (a teljes listában ugyanilyen objektumok szerepelnek egy tömbben)


```
{
  "id": 2,
  "name": "Etióp Yirgacheffe 250 g",
  "description": "Virágos, citrusos jegyek, világos pörkölés.",
  "price": 4290,
  "stock": 18,
  "categories": [
    {
      "id": 6,
      "name": "Ajándékötlet"
    },
    {
      "id": 1,
      "name": "Szemes kávé"
    }
  ]
}
```


POST backend/termekek.php – kérés és válasz (201):

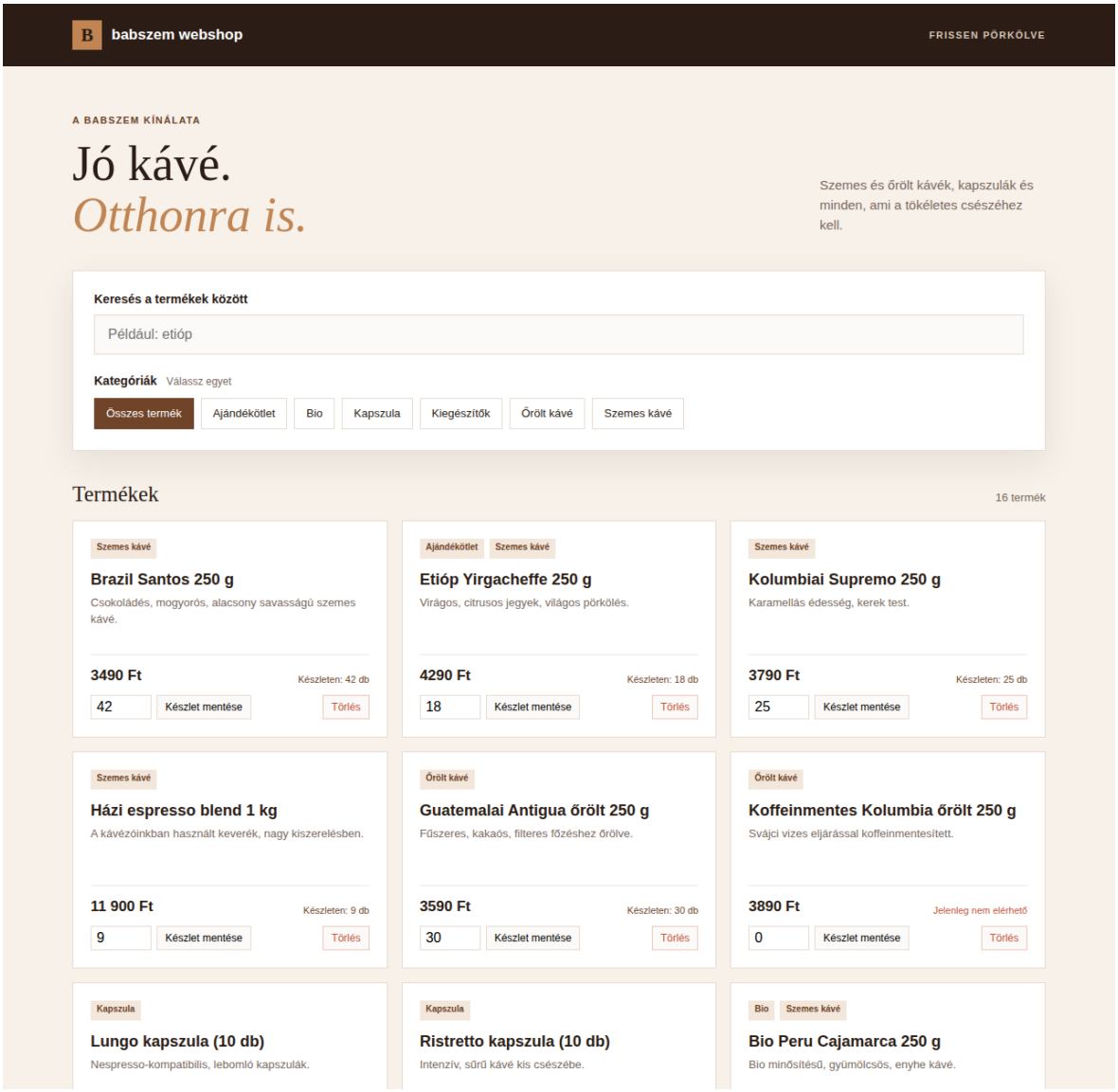

```
{
  "name": "Kenyai AA 250 g",
  "description": "Feketeribizlis, élénk savasság.",
  "price": 4590,
  "stock": 20,
  "categories": [1, 5]
}
```

```
{
  "message": "Termék sikeresen hozzáadva.",
  "id": 17
}
```


PATCH backend/termekek.php?id=6 törzs: {"stock": 15} → 200; valamint néhány hibaválasz (400, 404, 405):


```
{
  "message": "Készlet frissítve.",
  "id": 6,
  "stock": 15
}
{
  "error": "Az ár csak pozitív egész szám lehet."
}
{
  "error": "A termék nem található."
}
{
  "error": "Method not allowed"
}
```

A kész frontend a működő backenddel



A kártyák alján lévő mezővel és gombokkal a készlet módosítható (PATCH) és a termék törölhető (DELETE); az oldal alján lévő űrlappal új termék vehető fel (POST). Ha a backend hibát ad vissza, a frontend a hibaüzenetet kiírja az űrlap alá.

Melléklet – a mintamegoldás használata

A **Megoldás** mappában mindhárom rész egy lehetséges, működő megoldása található. Először próbálja meg önállóan megoldani a feladatokat, és csak ellenőrzésre használja!

```
Megoldás/
├── 1_API/BabszemAPI/          ← Visual Studio / dotnet projekt (Controllers, Models, Program.cs)
├── 2_Webdizajn/               ← index.html, style.css, script.js, kepek/
└── 3_PHP/babszem_webshop/
    ├── backend/               ← config.php, kategoriak.php, termekek.php
    └── frontend/              ← ugyanaz, mint a Forrásokban
```

- **API:** az *appsettings.json*-ban szükség esetén írja át a szerver nevét (pl. *Server=.\SQLEXPRESS*), majd futtassa a projektet (`dotnet run` vagy F5). A Swagger felület a `http://localhost:5000/swagger` címen nyílik meg.
- **Webdizájn:** nyissa meg az *index.html*-t böngészőben (vagy VS Code Live Serverrel), és a fejlesztői eszközök (F12) eszköztárával próbálja ki a különböző szélességeket.
- **PHP:** másolja a *babszem_webshop* mappát a *htdocs*-ba, importálja az adatbázist, majd nyissa meg a frontend oldalt. Egy törlés vagy új termék után az adatbázis a .sql fájl újraimportálásával állítható vissza.